

1. 강의를 통해 배운 점

이번 강의를 통해 AI 모델이 구동되는 전체 과정을 수학적·통계적 기초부터 데이터 전처리, 모델 학습, 그리고 실제 온디바이스 배포 및 경량화까지 체계적으로 학습하였다. 선형대수의 벡터와 행렬은 데이터를 표현하는 수단이며, AI 모델의 기본 연산은 입력값 $\times$ 가중치 $+$ 편향(bias)의 결합으로 이루어진다. 고차원 데이터의 차원 축소를 위해 주성분 분석(PCA)을 활용하며, 데이터 분산이 가장 큰 방향인 고유벡터와 그 방향의 중요도를 나타내는 고유값을 통해 중요한 정보를 보존하며 데이터를 압축하는 원리를 이해하였다. 통계 영역에서는 평균, 편차, 분산, 표준편차를 통해 데이터 분포를 해석하고, 정규화와 표준화를 통해 피처의 스케일을 맞춰 학습을 안정화하는 전처리 과정을 익혔다. 더불어 p-value를 통해 실험 결과의 통계적 유의성을 판단하는 방법과 상관관계와 인과관계의 차이를 파악하였다.

머신러닝과 딥러닝의 핵심 구조적 차이는 피처 추출 방식에 있다. 머신러닝은 인간이 직접 피처를 설계하지만, 딥러닝은 모델이 피처를 스스로 학습한다. 모델의 학습은 손실함수를 통해 현재의 오류를 정의하고, 경사하강법을 활용하여 손실을 최소화하는 방향으로 가중치를 수정하는 과정이다. 입력에 가중치를 곱하고 편향을 더한 뒤 활성화 함수를 통과시키는 퍼셉트론을 다층으로 쌓음으로써 단층 퍼셉트론으로 풀 수 없는 비선형 문제를 해결할 수 있음을 배웠다. 특히 이미지 처리 분야에서는 공간 구조를 반영하는 CNN의 합성곱(convolution)과 풀링(pooling) 연산 메커니즘을 학습하였으며, 모델의 일반화 성능을 높이기 위해 평가 데이터가 아닌 학습 데이터에만 데이터 증강을 적용해야 하는 이유를 이해하였다.

실전 응용 단계에서는 생성 모델의 활용과 온디바이스 AI 배포 조건을 다루었다. 오토인코더와 VAE는 반도체 공정과 같이 결함 데이터가 부족한 환경에서 이상 탐지 및 가상 데이터 생성에 활용될 수 있으나, 생성 데이터의 다양성과 물리적 타당성을 검증해야 한다는 한계도 명확히 인지하였다. 최종 단계인 온디바이스 AI 환경에서는 하드웨어 자원이 제한되므로 정확도뿐만 아니라 지연시간(latency), 메모리, 전력, 모델 크기 사이의 트레이드오프를 반드시 고려해야 한다. 이를 최적화하기 위해 양자화, 프루닝, 지식 증류 기술을 적용하고 TFLite로 변환하여 최종 장치에서 효율적으로 추론을 수행하는 전체 흐름을 정립하였다.

2. 개인 과제 및 실습 진행 공유

실제 라즈베리파이(Raspberry Pi) 환경에서 가위바위보 손 모양 탐지를 위한 SSD(Single Shot Multibox Detector) 오브젝트 디텍션 모델을 구동하고, 원본 Keras 모델(.h5)을 float32 및 int8 형식의 TFLite 모델로 변환하여 성능을 벤치마크했다.

양자화 전후 벤치마크 결과 비교

라즈베리파이 보드에서 benchmark_rps_ssd_lite_tflite.py 스크립트를 직접 실행하여 수집한 데이터는 다음과 같다.

평가 항목	FP32 SSD-Lite (rps_ssd_lite_fp32.tflite)	INT8 SSD-Lite (rps_ssd_lite_int8.tflite)
모델 파일 크기	115,876 Bytes	36,768 Bytes
입출력 데이터 타입	float32	int8
추론 시간(평균값)	1.1438 ms	1.2811 ms
추론 시간(중간값)	1.2299 ms	1.2771 ms
추론시간(최솟값)	0.8784 ms	1.2747 ms
추론시간(최댓값)	1.5283 ms	1.3476 ms
CPU 사용률	250.0%	190.0%
RSS 메모리	42.30 MB	42.09 MB

RSS 메모리가 거의 동일한 원인 분석:

경량화 파라미터를 적용했음에도 프로세스가 점유하는 실제 물리 메모리 크기인 RSS(Resident Set Size)는 약 42.30 MB에서 42.09 MB로 차이가 미미했다. 이는 전체 RSS가 아래와 같은 구조를 갖기 때문이다.

전체 RSS = 실행 환경 기본 메모리 + 모델 메모리 + 입출력 연산 버퍼

Python 런타임, OpenCV, NumPy, TFLite Runtime 라이브러리 자체가 점유하는 기본 프레임워크 메모리가 수십 MB 수준으로 큰 비중을 차지한다. 따라서 모델의 크기가 수십 KB 수준으로 매우 작을 때는 모델 크기를 줄여도 전체 RSS의 유의미한 하락으로 이어지지 않는다는 정량적 특성을 체득했다.

연산 효율 및 자원 소모의 대안적 가치:

속도는 다소 느려졌으나 CPU 사용률이 250.0%에서 190.0%로 크게 감소했다. 이는 저전력 연산이 필요하거나 다중 스레드가 동시에 구동되는 실시간 제어 임베디드 시스템에서 자원 할당 효율을 높이고 전력 소모를 낮추는 목적으로 양자화 기술을 전략적으로 선택할 수 있음을 시사한다.